// Day 15 of 30

CHEATSHEET

Design Patterns Backend Devs Use Without Knowing

→ Swipe through all 7 slides

Shubham Bhati
linkedin.com/in/bhatishubham

Singleton — @Service

You're already using it

```
@Service
public class UserService { }

// Spring creates ONE instance
// Shared across entire app
// That's the Singleton pattern!

Why: Save memory, share state
Where: Services, configs, connections
```

💡 Every @Service, @Component, @Repository is a Singleton

Shubham Bhati
linkedin.com/in/bhatishubham

Factory — @Bean

Object creation without 'new'

```
@Bean
public RestTemplate restTemplate() {
  return new RestTemplate();
}

// You define HOW to create it
// Spring decides WHEN to create it
// That's the Factory pattern!
```

💡 @Bean methods are factory methods

Proxy — @Transactional

Invisible wrapper around your code

```
@Transactional
public void transfer(a, b, amount) {
  debit(a, amount);
  credit(b, amount);
}

// Spring creates a PROXY class
// Wraps your method with:
//   begin transaction
//   your code
//   commit (or rollback)
```

💡 AOP, @Transactional, @Cacheable all use Proxy pattern

Shubham Bhati
linkedin.com/in/bhatishubham

Observer — @EventListener

React to events without coupling

```
// Publisher
eventPublisher.publishEvent(
  new OrderCreatedEvent(order));

// Listener (different class)
@EventListener
void onOrderCreated(OrderCreatedEvent e){
  sendEmail(e.getOrder());
}

// Publisher doesn't know listener exists
```

💡 Decouples components. Spring Events = Observer pattern.

Strategy — Interfaces

Swap behavior at runtime

```
interface PaymentStrategy {
  void pay(double amount);
}

@Service class CreditCardPay implements ...
@Service class UPIPay implements ...

// Inject the right one based on context
// No if-else chains needed
```

💡 Interface + multiple implementations = Strategy

Builder — Lombok @Builder

Clean object construction

```
@Builder
public class User {
  String name;
  String email;
  int age;
}

User u = User.builder()
  .name("Shubham")
  .email("shubham@test.com")
  .age(25)
  .build();
```

💡 Builder avoids constructors with 10 params

Template Method — JpaRepository

Framework defines structure, you fill gaps

```
public interface UserRepo
  extends JpaRepository {

  // Framework handles:
  //   findAll, findById, save, delete

  // YOU add custom queries:
  List findByEmail(String email);
}

// That's Template Method pattern
```

💡 Spring Data = Template Method in action

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham